

Nearest Neighbour Algorithms

Christos Dimitrakakis

August 14, 2026

Outline

Introduction

The hidden secret of machine learning

The algorithm

k Nearest Neighbours

Extensions and parameters

Activities

Activity 1: kNN in practice

ACTIVITY

Activity 2: Deciding, not just predicting

ACTIVITY

Pen-and-paper exercise

EXERCISE

Introduction

The hidden secret of machine learning

The algorithm

k Nearest Neighbours

Extensions and parameters

Activities

Activity 1: kNN in practice

ACTIVITY

Activity 2: Deciding, not just predicting

ACTIVITY

Pen-and-paper exercise

EXERCISE

Supervised learning

- ▶ Given labelled training examples $(x_1, y_1), \dots, (x_T, y_T)$ where
- ▶ $x_t \in X$ are **features**
- ▶ $y_t \in Y$ are **labels**..

Feature space $\mathcal{X}$

- ▶ Usually $\mathcal{X} = \mathbb{R}^n$: the n-dimensional Euclidean space
- ▶ How do we use your class data?

Classification

- ▶ $Y = \{1, \dots, m\}$ are **discrete** labels

Regression

- ▶ $Y = \mathbb{R}^m$ are **continuous** values

The kNN algorithm idea

- ▶ Assume an unknown example is similar to its neighbours
- ▶ Smoothness allows us to make predictions

Discriminatory analysis-nonparametric discrimination: consistency properties, Evelyn Fix and Joseph L. Hodges Jr, 1951.

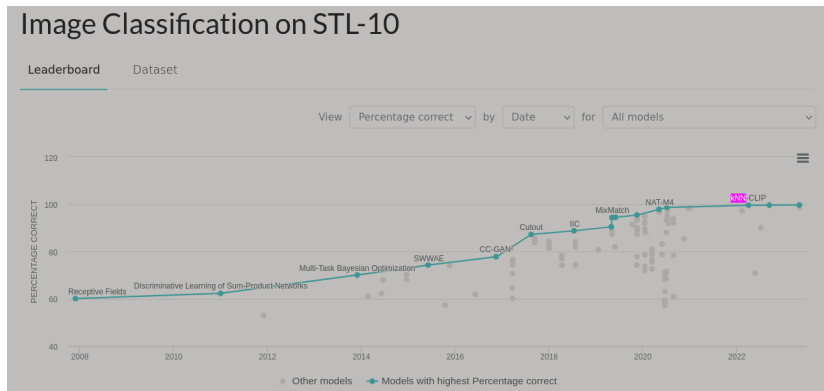


Figure: Evelyn Fix



Figure: Joseph Hodges

Performance of KNN on image classification



- ▶ Really simple!
- ▶ Can outperform really complex models!

Introduction

The hidden secret of machine learning

The algorithm

k Nearest Neighbours

Extensions and parameters

Activities

Activity 1: kNN in practice

ACTIVITY

Activity 2: Deciding, not just predicting

ACTIVITY

Pen-and-paper exercise

EXERCISE

The Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d

The Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d
- ▶ $t^* = \arg \min_t d(x_t, x)$ / How do we implement this?

The Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d
- ▶ $t^* = \arg \min_t d(x_t, x)$ / How do we implement this?
- ▶ Return $\hat{y}_t = y_{t^*}$

The Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d
- ▶ $t^* = \arg \min_t d(x_t, x)$ / How do we implement this?
- ▶ Return $\hat{y}_t = y_{t^*}$

The Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d
- ▶ $t^* = \arg \min_t d(x_t, x)$ / How do we implement this?
- ▶ Return $\hat{y}_t = y_{t^*}$

Classification

$$\hat{y}_t \in [m] \equiv \{1, \dots, m\}$$

The Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d
- ▶ $t^* = \arg \min_t d(x_t, x)$ / How do we implement this?
- ▶ Return $\hat{y}_t = y_{t^*}$

Classification

$$\hat{y}_t \in [m] \equiv \{1, \dots, m\}$$

Regression

$$\hat{y}_t \in \mathbb{R}^m$$

The k -Nearest Neighbour algorithm

Pseudocode

- Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d , neighbours k

The k -Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d , neighbours k
- ▶ Calculate $h_t = d(x_t, x)$ for all t .

The k -Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d , neighbours k
- ▶ Calculate $h_t = d(x_t, x)$ for all t .
- ▶ Get sorted indices $s = \text{argsort}(h)$ so that $d(x_{s_i}, x) \leq d(x_{s_{i+1}}, x)$ for all i .

The k -Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d , neighbours k
- ▶ Calculate $h_t = d(x_t, x)$ for all t .
- ▶ Get sorted indices $s = \text{argsort}(h)$ so that $d(x_{s_i}, x) \leq d(x_{s_{i+1}}, x)$ for all i .
- ▶ Return $\sum_{i=1}^k y_{s_i} / k$.

The k -Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d , neighbours k
- ▶ Calculate $h_t = d(x_t, x)$ for all t .
- ▶ Get sorted indices $s = \text{argsort}(h)$ so that $d(x_{s_i}, x) \leq d(x_{s_{i+1}}, x)$ for all i .
- ▶ Return $\sum_{i=1}^k y_{s_i} / k$.

The k -Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d , neighbours k
- ▶ Calculate $h_t = d(x_t, x)$ for all t .
- ▶ Get sorted indices $s = \text{argsort}(h)$ so that $d(x_{s_i}, x) \leq d(x_{s_{i+1}}, x)$ for all i .
- ▶ Return $\sum_{i=1}^k y_{s_i} / k$.

Classification

- ▶ We use a **one-hot encoding** $(0, \dots, 0, 1, 0, \dots, 0)$, with $y_t \in \{0, 1\}^m$.
- ▶ The class of the t -th example is $j \Leftrightarrow y_{t,j} = 1$.
- ▶ Equivalently, return p with

$$p_i = \sum_{t=1}^k \mathbb{I}\{y_{s_t} = i\} / k$$

The k -Nearest Neighbour algorithm

Pseudocode

- ▶ Input: Data $(x_t, y_t)_{t=1}^T$, test point x , distance d , neighbours k
- ▶ Calculate $h_t = d(x_t, x)$ for all t .
- ▶ Get sorted indices $s = \text{argsort}(h)$ so that $d(x_{s_i}, x) \leq d(x_{s_{i+1}}, x)$ for all i .
- ▶ Return $\sum_{i=1}^k y_{s_i} / k$.

Classification

- ▶ We use a **one-hot encoding** $(0, \dots, 0, 1, 0, \dots, 0)$, with $y_t \in \{0, 1\}^m$.
- ▶ The class of the t -th example is $j \Leftrightarrow y_{t,j} = 1$.
- ▶ Equivalently, return p with

$$p_i = \sum_{t=1}^k \mathbb{I}\{y_{s_t} = i\} / k$$

Regression

- ▶ $y_t \in \mathbb{R}^m$, so we need do nothing

The number of neighbours

$$k = 1$$

- ▶ How does it perform on the training data?
- ▶ How might it perform on unseen data?

$$k = T$$

- ▶ How does it perform on the training data?
- ▶ How might it perform on unseen data?

Distance function

For data in $\mathbb{R}^n$, p -norm

$$d(x, y) = \|x - y\|_p$$

Scaled norms

When features having varying scales:

$$d(x, y) = \|Sx - Sy\|_p$$

Or pre-scale the data

Complex data

- ▶ Manifold distances
- ▶ Graph distance

Distances

A distance $d(\cdot, \cdot)$:

- ▶ Identity $d(x, x) = 0$.
- ▶ Positivity $d(x, y) > 0$ if $x \neq y$.
- ▶ Symmetry $d(y, x) = d(x, y)$.
- ▶ Triangle inequality $d(x, y) \leq d(x, z) + d(z, y)$.

For data in $\mathbb{R}^n$, ℓ_p -norm

$$d(x, y) = \|x - y\|_p$$

Norms;

A norm $\| \cdot \|$

- ▶ Zero element $\|0\| = 0$.
- ▶ Homogeneity $\|cx\| = c\|x\|$ for any scalar a .
- ▶ Triangle inequality $\|x + y\| \leq \|x\| + \|y\|$.

ℓ_p -norm

$$\|z\|_p = \left(\sum_i z_i^p \right)^{1/p}$$

Neighbourhood calculation

If we have T datapoints

Sort and top K .

- Requires $O(T \ln T)$ time

Use the Cover-Tree or KD-Tree algorithm

- Requires $O(cK \ln T)$ time.
- c depends on the data distribution.

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = P(y = i|x),$$

where $P(y = i|x)$ is the probability that y is i , given x .

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = P(y = i|x),$$

where $P(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = P(y = i|x),$$

where $P(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i P(y_t = i|x_t)$

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = P(y = i|x),$$

where $P(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i P(y_t = i|x_t)$
- ▶ We observe the actual label y_t .

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = P(y = i|x),$$

where $P(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i P(y_t = i|x_t)$
- ▶ We observe the actual label y_t .
- ▶ We **win** if $y_t = a_t$ and **lose** otherwise

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = P(y = i|x),$$

where $P(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i P(y_t = i|x_t)$
- ▶ We observe the actual label y_t .
- ▶ We **win** if $y_t = a_t$ and **lose** otherwise

Making a decision

kNN as a model

- ▶ Given features x , we get a vector p of class probabilities:

$$p_i = P(y = i|x),$$

where $P(y = i|x)$ is the probability that y is i , given x .

Decisions to maximise accuracy

At time t :

- ▶ We observe features x_t
- ▶ We **predict** label $a_t = \arg \max_i P(y_t = i|x_t)$
- ▶ We observe the actual label y_t .
- ▶ We **win** if $y_t = a_t$ and **lose** otherwise

The model versus the prediction

- ▶ The **model** P tells us the probability of different classes.
- ▶ When we **decide** what our prediction should be, we can **use** the model.
- ▶ We will use π to denote the **decision rule** or **policy**.

Decisions versus predictions

- ▶ We frequently need to make a **decision**, instead of just a **prediction**.
- ▶ Our **utility** function $U(y, a)$ represents our **preferences**.
- ▶ The space of **actions** A is *not identical* to the set of **labels** Y .

Decisions versus predictions

- ▶ We frequently need to make a **decision**, instead of just a **prediction**.
- ▶ Our **utility** function $U(y, a)$ represents our **preferences**.
- ▶ The space of **actions** A is *not identical* to the set of **labels** Y .

Minimise spam annoyance

What utility function would you use for the spam detection problem?

Utility	Pass	Flag	Trash
Normal			
Spam			
Virus			

Decisions versus predictions

- ▶ We frequently need to make a **decision**, instead of just a **prediction**.
- ▶ Our **utility** function $U(y, a)$ represents our **preferences**.
- ▶ The space of **actions** A is *not identical* to the set of **labels** Y .

Minimise spam annoyance

What utility function would you use for the spam detection problem?

Utility	Pass	Flag	Trash
Normal			
Spam			
Virus			

Classification decision to maximise expected utility

- ▶ Expected utility of a single decision

$$\mathbb{E}[U|a, x] = \sum_y P(y|x, a) U(y, a) = \sum_y P(y|x) U(y, a)$$

- ▶ The decision maximising expected utility

$$a^* = \arg \max_a \mathbb{E}[U|a, x]$$

Introduction

The hidden secret of machine learning

The algorithm

k Nearest Neighbours

Extensions and parameters

Activities

Activity 1: kNN in practice

ACTIVITY

Activity 2: Deciding, not just predicting

ACTIVITY

Pen-and-paper exercise

EXERCISE

Setup

- ▶ Use the class dataset from earlier (or another dataset given in class).
- ▶ Use scikit-learn's `KNeighborsClassifier` (or `KNeighborsRegressor` if your target is continuous).

Tasks

- ▶ Fit the model for several values of k (e.g. $k = 1, 3, 5, 15, T$), recording both **training** and **test** accuracy for each.
- ▶ Rescale the features (e.g. with `StandardScaler`) and repeat. Does the ranking of neighbours change? Does performance change?
- ▶ Plot training accuracy and test accuracy against k on the same axes.

Keep this plot — you'll want it for the pen-and-paper exercise below.

Level notes

- ▶ **BSc**: Focus on the shape of the curve you just plotted. In plain language, what does it tell you about "knowing too little" versus "knowing too much" from the data? You'll see this exact shape of curve again and again this semester: start building intuition for it now.
- ▶ **MSc**: kNN regression is quietly estimating something specific: $\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i$ is an estimator of the **conditional expectation** $\mathbb{E}[Y \mid X = x]$. Write down why averaging nearby y -values is a sensible way to estimate $\mathbb{E}[Y \mid X = x]$, and what happens to this estimator as $k \rightarrow \infty$ with $k/T \rightarrow 0$. (If conditional expectation feels shaky, this is the moment to fix that: it underlies almost everything from here on, and it's worth re-deriving rather than assuming.)

Setup

As a class, agree on a utility matrix $U(y, a)$ for the spam example from the slides, by filling in the table:

Utility	Pass	Flag	Trash
Normal			
Spam			
Virus			

Tasks

- ▶ Fit a kNN classifier and call `predict_proba()` to get $P(y | x)$ for each test point.
- ▶ Write a function `utilityScore(y, actions, U)` that takes the true labels y_t and chosen actions a_t , and returns $\sum_{t=1}^T U(y_t, a_t) / T$.
- ▶ Write a second function `utilityAction(probs, U)` that returns, for each test point, the action a^* maximising expected utility under $P(y | x)$ and your U .
- ▶ Compare the average utility of plain `predict()` (most likely class) against your `utilityAction()`. Which one wins? Does the answer depend on which U you chose?

Level notes

- ▶ **BSc:** The hard part of this exercise isn't the code — it's agreeing on the utility table. Notice how a vague goal ("stop spam") had to become numbers before anything could be computed. That translation step, turning a question into something measurable, is the scientific-method skill this course is built around.
- ▶ **MSc:** Once you have $a^* = \arg \max_a \sum_y P(y|x)U(y, a)$, argue that no other decision rule can achieve higher expected utility under your model. Then ask: what changes once $P(y|x)$ is **estimated** (e.g. via kNN) rather than known exactly? Which new source of error are you now exposed to that wasn't there

Do this at the end of the class, or at the start of the next one, *after* looking back over the slides. No coding, no specific dataset — just the symbols.

Behaviour of k

Without picking any numbers or a specific dataset:

- ▶ Argue why the **training** accuracy of k -NN is always exactly 100% when $k = 1$.
- ▶ Argue why training accuracy can only get **worse, or stay the same**, as k grows from 1 towards T . (Hint: what does the prediction become when $k = T$?)
- ▶ Does the same argument apply to **test** accuracy? Why, or why not? Check this against the plot you made in Activity 1.

Level notes

- ▶ **BSc**: This is a sanity-check exercise as much as a math one: good models should behave sensibly at extreme settings, and being able to predict that **before** running any code is a useful scientific habit.
- ▶ **MSc**: Think of this as a conditional probability. What is the estimated conditional probability, and how or why does it differ from the true one?

Deciding from a generic utility

Let $U(y, a)$ be an arbitrary utility matrix and $P(y | x)$ an arbitrary model.

- ▶ Write down the expected utility of action a , and the action a^* that maximises it.
- ▶ Show that if $U(y, a) = \mathbb{I}\{y = a\}$ — you only care about being exactly right — then a^* is the most likely class, $\arg \max_y P(y | x)$.
- ▶ Now construct your **own** utility matrix U over 3 classes and 3 actions, for which a^* is **not** the most likely class for some $P(y | x)$ you choose. In words, explain why your U creates this mismatch — and compare it to the utility table your class agreed on in Activity 2.

Level notes

- ▶ **BSc**: Constructing your own mismatched U and explaining **why** it creates a mismatch, in plain words, is the actual goal: not the algebra.
- ▶ **MSc**: Make sure your proof for the $U(y, a) = \mathbb{I}\{y = a\}$ case is a genuine proof, not "it's obvious": show the sum collapses to a single term per candidate a , and argue why that term is maximised exactly at the most likely class.